

Trabajo Práctico Especial 1

Gestión de Equipajes en Aeropuertos

9 de Septiembre de 2026



Foto de [Jon Matthews](#) en [Unsplash](#)

Índice

Índice.....	2
Objetivo.....	3
Introducción.....	3
Terminología.....	4
Servicios Remotos y Clientes.....	5
1. Servicio de Administración de Equipajes.....	5
1.1 Agregar un pier.....	5
1.2 Agregar un cart.....	5
1.3 Asignar un pier a un vuelo.....	6
2. Servicio de Gestión de Vuelos.....	6
2.1 Agregar un vuelo.....	6
2.2 Iniciar o finalizar el bag-drop de un vuelo.....	7
2.3 Encender estación de bag-drop.....	8
2.4 Apagar una estación de bag-drop.....	9
3. Servicio de Despacho de Equipajes.....	9
3.1 Consultar franquicia no utilizada.....	10
3.2 Despachar equipajes usando una estación de bag-drop.....	11
3.3 Consultar equipaje despachado.....	13
4. Servicio de Rampa.....	15
4.1 Asignar un cart a un pier o liberar la asignación.....	16
4.2 Trasladar equipajes del pier al cart.....	16
4.3 Trasladar equipajes del cart al avión.....	17
5. Servicio de Seguimiento de Equipajes.....	18
5.1 Iniciar seguimiento de equipajes de un vuelo.....	18
5.2 Finalizar seguimiento de equipajes de un vuelo.....	20
5.3 Equipajes cargados en aviones.....	21
5.4 Equipajes pendientes de carga en aviones (Sólo para grupos de 5 integrantes).....	22
5.5 Dashboard de equipajes despachados de un vuelo (Sólo para grupos de 5 integrantes).	23
5.6 Dashboard de carts (Sólo para grupos de 5 integrantes).....	24
Requisitos.....	25
Informe.....	26
1. Decisiones de diseño e implementación de los servicios.....	26
2. Criterios aplicados para el trabajo concurrente.....	26
3. Potenciales puntos de mejora y/o expansión.....	26
Uso de Git.....	26
Material a entregar.....	26
Corrección.....	27
Cronograma.....	27
Dudas sobre el TPE.....	27
Recomendaciones.....	27

Objetivo

Implementar en grupos un **sistema remoto *multithreaded* y *thread-safe*** para la **gestión de equipajes de un aeropuerto** que cubra todo el recorrido de un equipaje desde que es despachado por el pasajero en una estación de *bag-drop* hasta su carga en el avión correspondiente.

Introducción

En un aeropuerto existen estaciones de autoservicio para despacho de equipajes (***self-service bag-drop stations***) donde los pasajeros pueden identificarse mediante su pasaporte y el código del vuelo para despachar sus equipajes. El sistema administra el recorrido de cada equipaje desde su aceptación en una estación de *bag-drop* hasta su carga en el avión correspondiente.

Para que una estación de *bag-drop* acepte un equipaje es la aerolínea la que primero tiene que haber informado al aeropuerto del detalle del vuelo que despegará (***manifest***) indicando los pasaportes de los pasajeros y las franquicias para cada uno. La aerolínea también debe indicar desde qué momento **se inicia el *bag-drop* del vuelo** de forma que los pasajeros pueden utilizar las estaciones de *bag-drop*.

Al despachar un equipaje se pesa y si está dentro de la franquicia se emite un ***bag-tag*** de confirmación y el equipaje viaja hasta la zona de equipajes (***baggage room***) que consiste en una cinta transportadora circular (***conveyor belt***).

Los equipajes del *baggage room* se mantienen allí hasta que el aeropuerto asigna un puerto (***pier***) a un vuelo. Así de forma automática y en función de la información indicada en el *bag-tag* cada equipaje es trasladado desde el *baggage room* al *pier* correspondiente y los nuevos equipajes de ese vuelo aceptados por las estaciones de *bag-drop* ahora irán directo al *pier*. Un vuelo no puede contar con más de un *pier* asociado.

Un carrito (***cart***) es el encargado de trasladar equipajes desde un *pier* hacia el avión. Primero se asigna un *cart* a un *pier*. Luego el personal de rampas traslada equipajes del *pier* al *cart*. Los *carts* cuentan con una capacidad máxima en kg de equipajes que pueden transportar. Después el personal de rampas realiza el traslado de equipajes desde el *cart* al avión. Una vez vacío, el *cart* puede realizar sucesivos viajes para el mismo *pier* y vuelo. Por último, se puede liberar su asignación para que quede disponible para ser asignado nuevamente.

Cuando la aerolínea decide **finalizar el *bag-drop*** (porque se acerca el momento del despegue) las estaciones de *bag-drop* ya no aceptan más equipajes de ese vuelo, todos los equipajes ya deben haber salido del *pier* y así ese *pier* libre queda disponible para ser asignado a otro vuelo.

El pasajero en todo momento puede consultar el estado de su equipaje despachado utilizando el *bag-tag* y así conocer si está en el *baggage room*, en un *pier*, en un *cart* o ya está en el avión.

Por su parte el supervisor de operaciones de equipajes del aeropuerto puede iniciar un seguimiento para recibir información sobre todos los eventos relacionados al manejo del equipaje de un vuelo y finalizarlo cuando lo desee. Además se pueden solicitar reportes varios sobre los equipajes despachados.

Terminología



Self-service bag-drop station

[Airport check-in - Wikipedia](#)

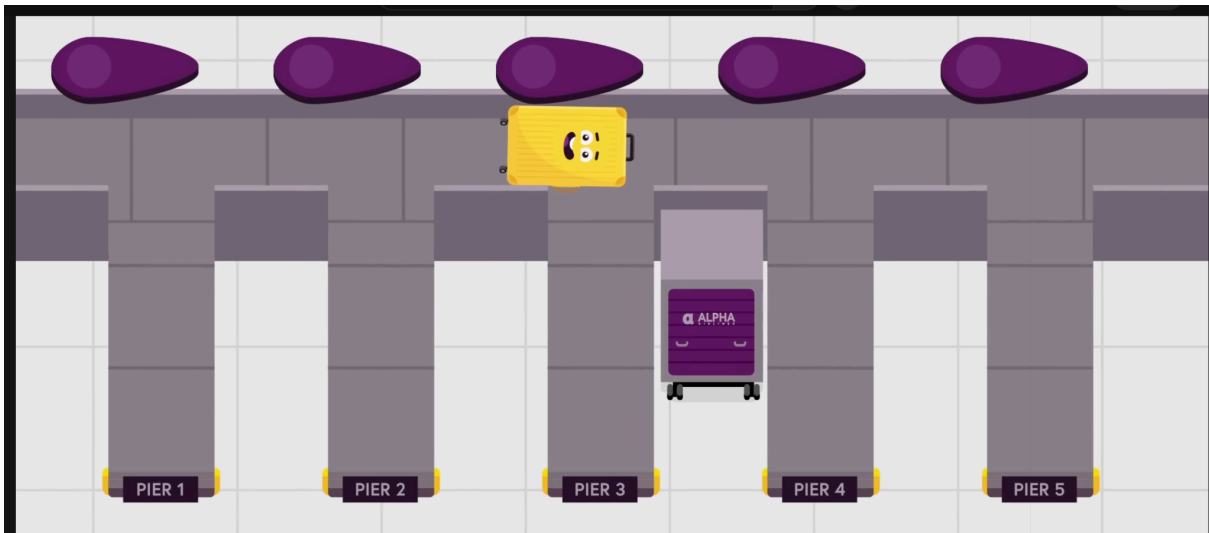
Foto: [Amadeus Seamless Bag Drop Series 7 - Airports](#)



Bag-tag

[Bag tag - Wikipedia](#)

Foto: [Checked baggage - Wikipedia](#)



Pier

Foto: [What Exactly Happens When You Check a Bag](#)



Conveyor belt

Foto: [Conveyor belt - Wikipedia](#)



Cart

Foto: [Baggage handler - Wikipedia](#)

Servicios Remotos y Clientes

El sistema requiere el desarrollo de los **siguientes servicios remotos con sus respectivos clientes**.

Es importante destacar que, aunque en este enunciado se mencionan servicios y clientes de manera conjunta por motivos de claridad y brevedad, **la implementación debe respetar la correcta separación entre servicio y cliente**. Para ello deben analizar si la lógica a implementar corresponde que esté del lado del servicio, del cliente o de ambos. Aunque en este trabajo práctico especial deberán implementar un único cliente de línea de comandos para cada uno de los servicios presentados considerar que un mismo servicio podría ser invocado por varios clientes implementados en diversas tecnologías (por ejemplo un cliente de línea de comandos y un cliente de una aplicación móvil).

1. Servicio de Administración de Equipajes

- Funcionalidad: Agregar *piers*, *carts* y asignar *piers* a vuelos.
- Usuario: Personal del Aeropuerto
- Cliente: La información de cuál es la acción a realizar se recibe a través de argumentos de línea de comando al llamar al *script* del cliente de administración de equipajes `bagAdmin.sh` y el resultado se debe imprimir en pantalla.

```
$> sh bagAdmin.sh -DserverAddress=xx.xx.xx.xx:yyyy
-Daction=(addPier|addCart|setPier) [ -DmaxWeight=weightInKg ] [
-Dpier=pierNumber -Dflight=flightCode ]
```

donde `xx.xx.xx.xx:yyyy` es la dirección IP y el puerto donde está publicado el servicio de administración de equipajes.

1.1 Agregar un *pier*

- Funcionalidad: Agregar un *pier*. Se indica el número de *pier* agregado. El *pier* *inicia* sin equipajes y sin vuelo asignado. La numeración de los *piers* es incremental donde el primero tiene el número 1.
- No falla
- Ejemplo de cliente de invocación: Agregar el *pier* "1"

```
$> sh bagAdmin.sh -DserverAddress=10.6.0.1:50051 -Daction=addPier
Added Pier #1
```

1.2 Agregar un *cart*

- Funcionalidad: Agregar un *cart* a partir del peso máximo en kg que puede transportar. Se indica el número de *cart* agregado. El *cart* *inicia* sin equipajes, sin *pier* asignado y sin vuelo asignado. La numeración de los *carts* es incremental donde el primero tiene el número 1.
- Falla si:
 - El peso máximo indicado no es un entero positivo
- Ejemplo de cliente de invocación: Agregar el *cart* "1" que puede transportar hasta 500 kg inclusive.


```
$> sh bagAdmin.sh -DserverAddress=10.6.0.1:50051 -Daction=addCart
-DmaxWeight=500
Added Cart #1
```

1.3 Asignar un *pier* a un vuelo

- Funcionalidad: Asignar un *pier* a un vuelo a partir del número del *pier* y del código del vuelo para que los equipajes correspondientes al vuelo que ya están en el *baggage room* se trasladen al *pier* y también para que de ahora en más los equipajes correspondientes al vuelo que son aceptados por las estaciones de *bag-drop* vayan directamente al *pier*.
- Falla si:
 - No existe un *pier* con ese número
 - No existe un vuelo con ese código
 - El *pier* ya tiene un vuelo asignado
 - El vuelo ya tiene un *pier* asignado
 - Ya finalizó el *bag-drop* del vuelo (Se invocó a 2.2 con END).
- Ejemplo de cliente de invocación: Asignar el *pier* 1 al vuelo de código AC1234

```
$> sh bagAdmin.sh -DserverAddress=10.6.0.1:50051 -Daction=setPier -Dpier=1
-Dflight=AC1234
Assigned Pier #1 to Flight AC1234
```

2. Servicio de Gestión de Vuelos

- Funcionalidad: Agregar vuelos, iniciar o finalizar el *bag-drop* de los vuelos y encender y apagar estaciones de *bag-drop*.
- Usuario: Personal de la Aerolínea
- Cliente: La información de cuál es la acción a realizar se recibe a través de argumentos de línea de comando al llamar al *script* del cliente de gestión de vuelos **flightManagement.sh** y el resultado se debe imprimir en pantalla.

```
$> sh flightManagement.sh -DserverAddress=xx.xx.xx.xx:yyyy
-Daction=(addFlight|setBagDrop|turnOnStation|turnOffStation) [
-Dflight=flightCode -Dmanifest=manifestDetail ] [ -Dflight=flightCode
-Dstatus=START|END ] [ -Dstation=stationUUID ]
```

donde xx.xx.xx.xx:yyyy es la dirección IP y el puerto donde está publicado el servicio de gestión de vuelos.

2.1 Agregar un vuelo

- Funcionalidad: Agregar un vuelo a partir del código del vuelo y el detalle de todos los pasajeros del vuelo con sus respectivas franquicias. Este detalle consiste, para cada pasajero, en el número de pasaporte y la franquicia del mismo. La franquicia de un pasajero consiste en la cantidad máxima de equipajes y el peso máximo en kg para cada uno de los equipajes. Todos los equipajes de un pasajero tienen el mismo peso máximo. El vuelo se agrega sin *pier* asignado y con el *bag-drop* no iniciado. Se indica el total de

72.42 Programación de Objetos Distribuidos - TPE1 2026 1C

pasajeros del vuelo, la cantidad total de equipajes que se pueden despachar y la suma de los pesos máximos en kg para todos esos equipajes.

- Falla si:
 - El vuelo no tiene pasajeros
 - Existen dos o más pasajeros con el mismo pasaporte
 - La cantidad máxima de equipajes de un pasajero no es un número natural
 - El peso máximo en kg para cada uno de los equipajes de un pasajero no es un entero positivo
 - Ya se agregó un vuelo con ese código
- Ejemplo de cliente de invocación: Agregar el vuelo de código AC1234 que cuenta con 3 pasajeros donde el pasajero con pasaporte ABC tiene una franquicia de 2 equipajes de hasta 23 kg cada uno y los demás (pasajeros con pasaportes DEF y HIJ) tienen una franquicia de 1 equipaje de hasta 27 kg cada uno. En la cadena de caracteres el símbolo “%” actúa como separador de pasajeros y el “@” como separador de campos para un pasajero.

ABC@2@23%DEF@1@27%HIJ@1@27

Pasajero 1 Pasajero 2 Pasajero 3

- **Número de pasaporte**
- **Cantidad máxima de equipajes**
- **Peso máximo en kg para cada uno de los equipajes**

```
$> sh flightManagement.sh -DserverAddress=10.6.0.1:50051 -Daction=addFlight  
-Dflight=AC1234 -Dmanifest=ABC@2@23%DEF@1@27%HIJ@1@27  
Flight AC1234 added with 3 passengers and up to 4 bags (100 kg)
```

2.2 Iniciar o finalizar el *bag-drop* de un vuelo

- Funcionalidad: Iniciar (START) o finalizar (END) el *bag-drop* de un vuelo a partir del código del vuelo. Se indica el estado luego de la operación, el total de pasajeros del vuelo, la cantidad total de equipajes que se pueden despachar y la suma de los pesos máximos en kg para todos esos equipajes.
 - Al iniciar el *bag-drop* de un vuelo las estaciones de *bag-drop* pueden aceptar equipajes del vuelo (invocaciones a 3.2)
 - Al finalizar el *bag-drop* de un vuelo las estaciones de *bag-drop* ya no pueden aceptar equipajes del vuelo (invocaciones a 3.2) y el *pier* asignado al vuelo queda sin vuelo asignado. Los *carts* asignados al *pier* mantienen la asignación hasta que se invoque a 4.1 con UNASSIGN.
- Falla si:
 - No existe un vuelo con ese código
 - Se quiere finalizar el *bag-drop* de un vuelo y ya se había iniciado y se cumple alguna de las siguientes condiciones:
 - el vuelo no tiene un *pier* asignado (No se invocó a 1.3 para ese vuelo)
 - existen equipajes correspondientes al vuelo en el *baggage room*
 - el *pier* asignado al vuelo tiene equipajes

72.42 Programación de Objetos Distribuidos - TPE1 2026 1C

- Se quiere finalizar el *bag-drop* de un vuelo y no había iniciado o ya se había finalizado
- Se quiere iniciar el *bag-drop* de un vuelo y ya se había iniciado o finalizado
- Ejemplo de cliente de invocación:
 - Iniciar el *bag-drop* para el vuelo de código AC1234.

```
$> sh flightManagement.sh -DserverAddress=10.6.0.1:50051 -Daction=setBagDrop  
-Dflight=AC1234 -Dstatus=START  
Bag-Drop STARTED for Flight AC1234 with 3 passengers and up to 4 bags (100  
kg)
```

- Finalizar el *bag-drop* para el vuelo de código AC1234.

```
$> sh flightManagement.sh -DserverAddress=10.6.0.1:50051 -Daction=setBagDrop  
-Dflight=AC1234 -Dstatus=END  
Bag-Drop ENDED for Flight AC1234 with 3 passengers and up to 4 bags (100 kg)
```

2.3 Encender estación de *bag-drop*

- Funcionalidad: Encender una estación de autoservicio para despacho de equipajes (*self-service bag-drop station*) que acepte equipajes de pasajeros de cualquier vuelo donde se haya iniciado el *bag-drop* y no haya finalizado, indicando **en tiempo real** los *bag-tags* emitidos en cada aceptación (con invocaciones a 3.2). Se indica un identificador UUID v4 generado para distinguir una estación de *bag-drop* de otra.
- No falla
- Ejemplo de cliente de invocación: Se enciende una estación de *bag-drop* de UUID f9c592e0-fde3-46a9-a6b4-264ed4d1eff0 y se mantiene encendida hasta que se decida apagarla (invocando a 2.4).

```
$> sh flightManagement.sh -DserverAddress=10.6.0.1:50051  
-Daction=turnOnStation  
###  
Bag-Drop Station f9c592e0-fde3-46a9-a6b4-264ed4d1eff0 is ON  
###  
—
```

Cuando el pasajero de pasaporte ABC despacha un equipaje de 22.5 kg para el vuelo de código AC1234 utilizando esta estación de *bag-drop* se indica en la salida el *bag-tag* 412 (resultado de invocar a 3.2).

```
$> sh flightManagement.sh -DserverAddress=10.6.0.1:50051  
-Daction=turnOnStation  
###  
Bag-Drop Station f9c592e0-fde3-46a9-a6b4-264ed4d1eff0 is ON  
###  
Bag-Tag Details
```



```
-> Flight AC1234
-> Passenger ABC
-> Weight 22.5 kg
-> Bag-Tag #412
###
-
```

Más tarde el pasajero de pasaporte ZZZ despacha un equipaje de 20.3 kg para el vuelo de código OS987 utilizando esta estación de bag-drop y se indica en la salida el bag-tag 413.

```
$> sh flightManagement.sh -DserverAddress=10.6.0.1:50051
-Daction=turnOnStation
###
Bag-Drop Station f9c592e0-fde3-46a9-a6b4-264ed4d1eff0 is ON
###
Bag-Tag Details
-> Flight AC1234
-> Passenger ABC
-> Weight 22.5 kg
-> Bag-Tag #412
###
Bag-Tag Details
-> Flight OS987
-> Passenger ZZZ
-> Weight 20.3 kg
-> Bag-Tag #413
###
-
```

2.4 Apagar una estación de *bag-drop*

- Funcionalidad: Apagar una estación de *bag-drop* que esté encendida (se invocó a 2.3) a partir del identificador UUID v4 de la estación. Se indica el estado luego de la operación.
- Falla si:
 - No existe una estación de *bag-drop* encendida con ese UUID
 - La estación de *bag-drop* está siendo utilizada por otra operación de despacho que no finalizó (Se invocó a 3.2 pero no finalizó)
- Ejemplo de cliente de invocación: Desde la consola t2 apagar la estación de *bag-drop* encendida de UUID f9c592e0-fde3-46a9-a6b4-264ed4d1eff0 que se está ejecutando en la consola t1 (donde se invocó a 2.3), mostrando el evento de apagado en las dos consolas, y finalizando la ejecución de los dos clientes:

```
t1$> sh flightManagement.sh
-DserverAddress=10.6.0.1:50051
-Daction=turnOnStation
###
```

```
t2$> sh flightManagement.sh
-DserverAddress=10.6.0.1:50051
-Daction=turnOffStation
-Dstation=f9c592e0-fde3-46a9-a6b4-264
ed4d1eff0
```

<pre> Bag-Drop Station f9c592e0-fde3-46a9-a6b4-264ed4d1eff0 is ON ### ... Bag-Drop Station f9c592e0-fde3-46a9-a6b4-264ed4d1eff0 is OFF t1\$> _ </pre>	<pre> Bag-Drop Station f9c592e0-fde3-46a9-a6b4-264ed4d1eff0 is OFF t2\$> _ </pre>
--	--

3. Servicio de Despacho de Equipajes

- Funcionalidad: Consultar franquicia, despachar equipajes y consultar los equipajes despachados a partir de los *bag-tags*.
- Usuario: Pasajeros
- Cliente: La información de cuál es la acción a realizar se recibe a través de argumentos de línea de comando al llamar al *script* del cliente del servicio de despacho de equipajes **bagDrop.sh** y el resultado se debe imprimir en pantalla.

```

$> sh bagDrop.sh -DserverAddress=xx.xx.xx.xx:yyyy
-Daction=(remainingBags|bagDrop|checkBag) [ -Dflight=flightCode
-Dpassenger=passport] [ -Dflight=flightCode -Dpassenger=passport
-Dstation=stationUUID] [ -DbagTag=bagTagNumber -Dflight=flightCode
-Dpassenger=passport]

```

donde xx.xx.xx.xx:yyyy es la dirección IP y el puerto donde está publicado el servicio de despacho de equipajes.

3.1 Consultar franquicia no utilizada

- Funcionalidad: Consultar la franquicia no utilizada a partir del código del vuelo y el pasaporte del pasajero. Se indica si el *bag-drop* del vuelo no inició (NOT_STARTED), si ya inició (STARTED) o si ya finalizó (ENDED) y la franquicia no utilizada. La franquicia no utilizada consiste en la cantidad máxima de equipajes permitidos menos la cantidad ya despachada, junto con el peso máximo en kg por equipaje. La franquicia no utilizada se informa independientemente del estado del *bag-drop*.
- Falla si:
 - No existe un vuelo con ese código
 - No existe un pasajero con ese pasaporte para ese vuelo
- Ejemplo de cliente de invocación: Se consulta la franquicia no utilizada del pasajero de pasaporte ABC para el vuelo de código AC1234, donde no se inició aún el *bag-drop*. La franquicia consiste en dos equipajes de hasta 23 kg cada uno (tal como se indicó en 2.1).

```

$> sh bagDrop.sh -DserverAddress=10.6.0.1:50051 -Daction=remainingBags
-Dflight=AC1234 -Dpassenger=ABC
Bag-Drop for Flight AC1234 is NOT_STARTED
Remaining Bags: 2
Max Weight: 23 kg

```

72.42 Programación de Objetos Distribuidos - TPE1 2026 1C

Luego de iniciado el *bag-drop* (invocando a 2.2 con START)

```
$> sh bagDrop.sh -DserverAddress=10.6.0.1:50051 -Daction=remainingBags  
-Dflight=AC1234 -Dpassenger=ABC  
Bag-Drop for Flight AC1234 is STARTED  
Remaining Bags: 2  
Max Weight: 23 kg
```

Luego de haber despachado un equipaje (invocando a 3.2) se indica que de los dos equipajes posibles sólo queda uno por despachar

```
$> sh bagDrop.sh -DserverAddress=10.6.0.1:50051 -Daction=remainingBags  
-Dflight=AC1234 -Dpassenger=ABC  
Bag-Drop for Flight AC1234 is STARTED  
Remaining Bags: 1  
Max Weight: 23 kg
```

Al finalizar el bag-drop del vuelo (invocando a 2.2 con END)

```
$> sh bagDrop.sh -DserverAddress=10.6.0.1:50051 -Daction=remainingBags  
-Dflight=AC1234 -Dpassenger=ABC  
Bag-Drop for Flight AC1234 is ENDED  
Remaining Bags: 1  
Max Weight: 23 kg
```

3.2 Despachar equipajes usando una estación de *bag-drop*

- Funcionalidad: Despachar los equipajes de un pasajero de un vuelo utilizando una estación de *bag-drop* encendida a partir del código del vuelo, el pasaporte del pasajero y el identificador UUID de la estación de *bag-drop* encendida, donde se ingresa, de a un equipaje a la vez, el peso de cada equipaje a despachar. Se indica el *bag-tag* emitido por cada despacho y la franquicia no utilizada (cantidad de equipajes y peso máximo en kg de cada uno). La numeración de los *bag-tags* es incremental y compartida por todas las estaciones de *bag-drop* donde el primero tiene el número 1.
- Falla si:
 - No existe un vuelo con ese código
 - No existe un pasajero con ese pasaporte para ese vuelo
 - No existe una estación de *bag-drop* encendida con ese UUID
 - La estación de *bag-drop* ya está siendo utilizada por otra operación de despacho que no finalizó (Se invocó a 3.2 pero no finalizó)
 - La estación de *bag-drop* no acepta equipajes de ese vuelo porque el *bag-drop* del vuelo no inició o ya finalizó
 - El peso del equipaje no es un número real positivo o excede el peso máximo permitido por la franquicia para ese pasajero
 - Se intenta despachar un equipaje cuando el pasajero ya agotó su franquicia (Ver más abajo, el cliente interactivo no debe permitirlo)

72.42 Programación de Objetos Distribuidos - TPE1 2026 1C

- Ejemplo de cliente de invocación: El pasajero de pasaporte ABC del vuelo de código AC1234 utiliza la estación de *bag-drop* de UUID f9c592e0-fde3-46a9-a6b4-264ed4d1eff0 para empezar a despachar equipajes, indicando que puede despachar 2 de 23 kg cada uno.

```
$> sh bagDrop.sh -DserverAddress=10.6.0.1:50051 -Daction=bagDrop
-Dflight=AC1234 -Dpassenger=ABC
-Dstation=f9c592e0-fde3-46a9-a6b4-264ed4d1eff0
###
Bag-Drop Station f9c592e0-fde3-46a9-a6b4-264ed4d1eff0
###
Bag-Drop for Flight AC1234
Remaining Bags: 2
Max Weight: 23 kg
###
Enter Weight of Bag or EXIT: _
```

Despacha el primer equipaje indicando un peso de 22.5 kg

```
$> sh bagDrop.sh -DserverAddress=10.6.0.1:50051 -Daction=bagDrop
-Dflight=AC1234 -Dpassenger=ABC
-Dstation=f9c592e0-fde3-46a9-a6b4-264ed4d1eff0
###
Bag-Drop Station f9c592e0-fde3-46a9-a6b4-264ed4d1eff0
###
Bag-Drop for Flight AC1234
Remaining Bags: 2
Max Weight: 23 kg
###
Enter Weight of Bag or EXIT: 22.5
```

y al realizarse la operación obtiene el *bag-tag* 412 y ya puede despachar el segundo equipaje (indicando su peso) o finalizar la operación (indicando EXIT) si no desea despachar otro equipaje.

```
$> sh bagDrop.sh -DserverAddress=10.6.0.1:50051 -Daction=bagDrop
-Dflight=AC1234 -Dpassenger=ABC
-Dstation=f9c592e0-fde3-46a9-a6b4-264ed4d1eff0
###
Bag-Drop Station f9c592e0-fde3-46a9-a6b4-264ed4d1eff0
###
Bag-Drop for Flight AC1234
Remaining Bags: 2
Max Weight: 23 kg
###
Enter Weight of Bag or EXIT: 22.5
```

```

Bag-Tag Details
-> Flight AC1234
-> Passenger ABC
-> Weight 22.5 kg
-> Bag-Tag #412
###
Bag-Drop for Flight AC1234
Remaining Bags: 1
Max Weight: 23 kg
###
Enter Weight of Bag or EXIT: _

```

Como no quiere despachar un segundo equipaje ingresa EXIT y finaliza el cliente.

```

$> sh bagDrop.sh -DserverAddress=10.6.0.1:50051 -Daction=bagDrop
-Dflight=AC1234 -Dpassenger=ABC
-Dstation=f9c592e0-fde3-46a9-a6b4-264ed4d1eff0
###
Bag-Drop Station f9c592e0-fde3-46a9-a6b4-264ed4d1eff0
###
Bag-Drop for Flight AC1234
Remaining Bags: 2
Max Weight: 23 kg
###
Enter Weight of Bag or EXIT: 22.5
Bag-Tag Details
-> Flight AC1234
-> Passenger ABC
-> Weight 22.5 kg
-> Bag-Tag #412
###
Bag-Drop for Flight AC1234
Remaining Bags: 1
Max Weight: 23 kg
###
Enter Weight of Bag or EXIT: EXIT
$>

```

Cuando no quedan equipajes por despachar el cliente debe mostrar 0 en “Remaining Bags” y finalizar la ejecución (de forma que el cliente no permite despachar más equipajes de los permitidos en la franquicia).

3.3 Consultar equipaje despachado

- Funcionalidad: Consultar el detalle de un equipaje despachado a partir del *bag-tag* (emitido por 3.2), el código del vuelo y el pasaporte del pasajero. Se indica si el *bag-drop* del vuelo ya inició (STARTED) o si ya finalizó (ENDED), el peso del equipaje y la ubicación del equipaje. Las ubicaciones posibles del equipaje son:

72.42 Programación de Objetos Distribuidos - TPE1 2026 1C

- En el *baggage room* porque todavía no hay pier asignado (no se invocó a 1.3)
- En el *pier* (se invocó a 1.3)
- En el *cart* (se invocó a 4.2)
- En el avión (se invocó a 4.3)

y se indican en pantalla todas las ubicaciones destacando con una [X] la ubicación actual.

- Falla si:
 - No existe un vuelo con ese código
 - No existe un pasajero con ese pasaporte para ese vuelo
 - No existe un *bag-tag* con ese número
 - El *bag-tag* no corresponde a ese pasajero o vuelo
- Ejemplo de cliente de invocación: El pasajero de pasaporte ABC consulta el primer equipaje despachado para el vuelo de código AC1234 donde el *bag-tag* es 412 y se indica que el equipaje pesó 22.5 kg y está en el *baggage room*.

```
$> sh bagDrop.sh -DserverAddress=10.6.0.1:50051 -Daction=checkBag
-DbagTag=412 -Dflight=AC1234 -Dpassenger=ABC
Bag-Drop for Flight AC1234 is STARTED
###
Bag-Tag Details
-> Flight AC1234
-> Passenger ABC
-> Weight 22.5 kg
-> Bag-Tag #412
###
Bag-Tag Location
[X] Baggage Room
[ ] Pier
[ ] Cart
[ ] Plane
```

Luego se le asigna el *pier* 1 al vuelo (invocando a 1.3) y se indica que el equipaje está en el *pier* 1.

```
$> sh bagDrop.sh -DserverAddress=10.6.0.1:50051 -Daction=checkBag
-DbagTag=412 -Dflight=AC1234 -Dpassenger=ABC
Bag-Drop for Flight AC1234 is STARTED
###
Bag-Tag Details
-> Flight AC1234
-> Passenger ABC
-> Weight 22.5 kg
-> Bag-Tag #412
###
Bag-Tag Location
[ ] Baggage Room
[X] Pier #1
[ ] Cart
```



```
[ ] Plane
```

Luego se traslada a este equipaje desde el *pier* 1 al *cart* 1 (invocando a 4.2)

```
$> sh bagDrop.sh -DserverAddress=10.6.0.1:50051 -Daction=checkBag
-DbagTag=412 -Dflight=AC1234 -Dpassenger=ABC
Bag-Drop for Flight AC1234 is STARTED
###
Bag-Tag Details
-> Flight AC1234
-> Passenger ABC
-> Weight 22.5 kg
-> Bag-Tag #412
###
Bag-Tag Location
[ ] Baggage Room
[ ] Pier
[X] Cart #1
[ ] Plane
```

Luego se traslada a este equipaje desde el *cart* 1 al avión del vuelo (invocando a 4.3) y se finaliza el *bag-drop* del vuelo.

```
$> sh bagDrop.sh -DserverAddress=10.6.0.1:50051 -Daction=checkBag
-DbagTag=412 -Dflight=AC1234 -Dpassenger=ABC
Bag-Drop for Flight AC1234 is ENDED
###
Bag-Tag Details
-> Flight AC1234
-> Passenger ABC
-> Weight 22.5 kg
-> Bag-Tag #412
###
Bag-Tag Location
[ ] Baggage Room
[ ] Pier
[ ] Cart
[X] Plane
```

4. Servicio de Rampa

- Funcionalidad: Asignar un *cart* a un *pier*, liberar las asignaciones realizadas, trasladar equipajes de un *pier* a un *cart* y de un *cart* al avión.
- Usuario: Personal de rampas
- Cliente: La información de cuál es la acción a realizar se recibe a través de argumentos de línea de comando al llamar al *script* del cliente del servicio de rampa **ramp.sh** y el resultado se debe imprimir en pantalla.

```
$> sh ramp.sh -DserverAddress=xx.xx.xx.xx:yyyy
-Daction=(setCartStatus|loadCart|unloadCart) [ -Dflight=flightCode
-Dpier=pierNumber -Dcart=cartNumber -Dstatus=ASSIGN|UNASSIGN ] [
-Dpier=pierNumber -Dcart=cartNumber -DbagTags=bagTagsList ] [ -Dcart=cartNumber
-DbagTags=bagTagsList ]
```

donde xx.xx.xx.xx:yyyy es la dirección IP y el puerto donde está publicado el servicio de rampa.

4.1 Asignar un *cart* a un *pier* o liberar la asignación

- Funcionalidad: Asignar (ASSIGN) un *cart* a un *pier* para poder realizar la carga de los equipajes de ese *pier* o liberar la asignación (UNASSIGN) de un *cart*, a partir del código del vuelo, el número del *pier* y el número del *cart*. Se indica el estado luego de la operación.
- Falla si:
 - No existe un *cart* con ese número
 - No existe un *pier* con ese número
 - No existe un vuelo con ese código
 - Si se quiere asignar y se cumple alguna de las siguientes condiciones:
 - el *cart* ya está asignado a un *pier*
 - el *pier* no está asignado a ese vuelo
 - Si se quiere liberar la asignación y se cumple alguna de las siguientes condiciones:
 - el *cart* no está asignado al *pier* y vuelo
 - el *cart* tiene equipajes
- Ejemplo de cliente de invocación:
 - Asignar el *cart* 1 al *pier* 1 para los equipajes del vuelo de código AC1234

```
$> sh ramp.sh -DserverAddress=10.6.0.1:50051 -Daction=setCartStatus
-Dflight=AC1234 -Dpier=1 -Dcart=1 -Dstatus=ASSIGN
Cart #1 assigned to Pier #1 for Flight AC1234
```

- Liberar la asignación del *cart* 1

```
$> sh ramp.sh -DserverAddress=10.6.0.1:50051 -Daction=setCartStatus
-Dflight=AC1234 -Dpier=1 -Dcart=1 -Dstatus=UNASSIGN
Cart #1 is IDLE
```

4.2 Trasladar equipajes del *pier* al *cart*

- Funcionalidad: Trasladar equipajes desde el *pier* hacia un *cart* asignado a ese *pier* a partir del número del *pier*, el número del *cart* y el listado de los *bag-tags*. Se indica el peso actual del *cart*, el detalle de cada *bag-tag* trasladado y el peso actualizado luego de cargar cada uno de los equipajes en el *cart*. En la cadena de caracteres el símbolo "@" actúa como separador de *bag-tags*.
- Falla si:
 - No existe un *cart* con ese número
 - No existe un *pier* con ese número

72.42 Programación de Objetos Distribuidos - TPE1 2026 1C

- El *cart* no está asignado a ese *pier*
- El vuelo correspondiente al *cart* no coincide con el vuelo correspondiente al *pier*
- No existe un equipaje en el *pier* con alguno de los *bag-tags* indicados (y no se concreta el traslado de ninguno de los equipajes indicados).
- El *cart* excede el peso máximo permitido al trasladar todos los equipajes (y no se concreta el traslado de ninguno de los equipajes indicados).
- Ejemplo de cliente de invocación: Trasladar un conjunto de equipajes desde el *pier* 1 al *cart* 1 que soporta un peso máximo de 500 kg. El conjunto consiste en los *bag-tags* 412 (22.5 kg) y 416 (21.0 kg) correspondientes al vuelo de código AC1234.

```
$> sh ramp.sh -DserverAddress=10.6.0.1:50051 -Daction=loadCart -Dpier=1
-Dcart=1 -DbagTags=412@416
###
Cart #1 weighs 0.0 kg
###
Bag-Tag Details
-> Flight AC1234
-> Passenger ABC
-> Weight 22.5 kg
-> Bag-Tag #412
###
Cart #1 weighs 22.5 kg
###
Bag-Tag Details
-> Flight AC1234
-> Passenger DEF
-> Weight 21.0 kg
-> Bag-Tag #416
###
Cart #1 weighs 43.5 kg
###
```

4.3 Trasladar equipajes del *cart* al avión

- Funcionalidad: Trasladar equipajes desde el *cart* hacia el avión del vuelo asignado a ese *cart* a partir del número del *cart* y el listado de los *bag-tags*. Se indica el peso actual del *cart*, el detalle de cada *bag-tag* trasladado y el peso actualizado luego de descargar cada uno de los equipajes del *cart*. En la cadena de caracteres el símbolo “@” actúa como separador de *bag-tags*.
- Falla si:
 - No existe un *cart* con ese número
 - No existe un equipaje en el *cart* con alguno de los *bag-tags* indicados (y no se concreta el traslado de ninguno de los equipajes indicados).
- Ejemplo de cliente de invocación: Trasladar un conjunto de equipajes desde el *cart* 1 al avión del vuelo de código AC1234. El conjunto consiste en los *bag-tags* 412 (22.5 kg) y 416 (21.0 kg).

```
$> sh ramp.sh -DserverAddress=10.6.0.1:50051 -Daction=unloadCart -Dcart=1
-DbagTags=416@412
###
Cart #1 weighs 43.5 kg
###
Bag-Tag Details
-> Flight AC1234
-> Passenger DEF
-> Weight 21.0 kg
-> Bag-Tag #416
###
Cart #1 weighs 22.5 kg
###
Bag-Tag Details
-> Flight AC1234
-> Passenger ABC
-> Weight 22.5 kg
-> Bag-Tag #412
###
Cart #1 weighs 0.0 kg
###
```

5. Servicio de Seguimiento de Equipajes

- Funcionalidad: Iniciar el seguimiento de equipajes de un vuelo, finalizar el seguimiento y consultar el detalle de los equipajes despachados
- Usuario: Supervisor de operaciones de equipajes del aeropuerto
- Cliente: La información de cuál es la acción a realizar se recibe a través de argumentos de línea de comando al llamar al *script* del cliente del servicio de seguimiento de equipajes **track.sh** y el resultado se debe imprimir en pantalla.

```
$> sh track.sh -DserverAddress=xx.xx.xx.xx:yyyy
-Daction=(watchFlight|unwatchFlight|planeBags|pendingBags|dashboard|cartBags) [
-Dflight=flightCode ] [ -Dflight=flightCode ] [ -Dflight=flightCode ] [
-Dflight=flightCode ] [ -Dflight=flightCode ] [ -Dflight=flightCode ]
```

donde xx.xx.xx.xx:yyyy es la dirección IP y el puerto donde está publicado el servicio de seguimiento de equipajes.

5.1 Iniciar seguimiento de equipajes de un vuelo

- Funcionalidad: Iniciar el seguimiento de equipajes de un vuelo a partir del código del vuelo para recibir, **en tiempo real**, toda la información relacionada al despacho de equipajes del vuelo desde ese momento.
- Se considera un evento:
 - **Evento Inicial**: Se registró correctamente para recibir notificaciones
Started watching Flight AC1234
 - Se asignó un *pier* al vuelo (se invocó a 1.3)

72.42 Programación de Objetos Distribuidos - TPE1 2026 1C

Assigned Pier #1 to Flight AC1234

- Inició el *bag-drop* del vuelo (se invocó a 2.2 con START)
Bag-Drop STARTED for Flight AC1234 with 3 passengers and up to 4 bags (100 kg)
- Se despachó un equipaje del pasajero del vuelo utilizando una estación de *bag-drop* y quedó en el *baggage room* (se invocó a 3.2 pero no a 1.3)

####

Bag-Tag Details

-> Flight AC1234

-> Passenger ABC

-> Weight 22.5 kg

-> Bag-Tag #412

####

Bag-Tag Location

☒ Baggage Room

☐ Pier

☐ Cart

☐ Plane

####

- Se despachó un equipaje del pasajero del vuelo utilizando una estación de *bag-drop* y quedó en el *pier* (se invocó a 3.2 después de 1.3) o un equipaje se trasladó del *baggage room* al *pier* (se invocó a 1.3 después de 3.2) indicando para cada equipaje

####

Bag-Tag Details

-> Flight AC1234

-> Passenger ABC

-> Weight 22.5 kg

-> Bag-Tag #412

####

Bag-Tag Location

☐ Baggage Room

☒ Pier #1

☐ Cart

☐ Plane

####

- Se asignó un *cart* al *pier* del vuelo (se invocó a 4.1 con ASSIGN)

Cart #1 assigned to Pier #1 for Flight AC1234

- Se trasladaron equipajes del *pier* al *cart* (se invocó a 4.2), indicando para cada equipaje

####

Bag-Tag Details

-> Flight AC1234

-> Passenger ABC

-> Weight 22.5 kg

-> Bag-Tag #412

####

Bag-Tag Location

☐ Baggage Room

72.42 Programación de Objetos Distribuidos - TPE1 2026 1C

```
[ ] Pier
[X] Cart #1
[ ] Plane
####
```

- Se trasladaron equipajes del *cart* al avión (se invocó a 4.3), indicando para cada equipaje

```
####
Bag-Tag Details
-> Flight AC1234
-> Passenger ABC
-> Weight 22.5 kg
-> Bag-Tag #412
####
```

```
Bag-Tag Location
[ ] Baggage Room
[ ] Pier
[ ] Cart
[X] Plane
####
```

- Se liberó la asignación del *cart* (se invocó a 4.1 con UNASSIGN)

```
Cart #1 is IDLE
```

- Finalizó el *bag-drop* del vuelo (se invocó a 2.2 con END)

```
Bag-Drop ENDED for Flight AC1234 with 3 passengers and up to 4 bags
```

```
(100 kg)
```

- **Evento Final:** Finalizado el *bag-drop* del vuelo, cuando todos los equipajes aceptados por las estaciones de *bag-drop* están presentes en el avión se emite un evento final que finaliza la ejecución del cliente ya que no se obtendrán más eventos relevantes al vuelo (se descarta la liberación de asignación de los *carts*).

```
All bags in plane
```

- Falla si:

- No existe un vuelo con ese código
- Ya finalizó el *bag-drop* del vuelo y todos los equipajes aceptados por las estaciones de *bag-drop* están presentes en el avión

- Ejemplo de cliente de invocación: Iniciar el seguimiento de equipajes del vuelo de código AC1234 para recibir, en tiempo real, todos los eventos correspondientes desde ese momento. Se muestra el evento inicial, donde el cliente no termina y muestra los eventos a medida que van ocurriendo:

```
$> sh track.sh -DserverAddress=10.6.0.1:50051 -Daction=watchFlight
-Dflight=AC1234
Started watching Flight AC1234
Assigned Pier #1 to Flight AC1234
Bag-Drop STARTED for Flight AC1234 with 3 passengers and up to 4 bags (100
kg)
—
```

5.2 Finalizar seguimiento de equipajes de un vuelo

- Funcionalidad: Finalizar el seguimiento de equipajes de un vuelo a partir del código del vuelo para dejar de recibir la información relacionada. Finaliza la ejecución de todos los clientes de notificaciones correspondientes (invocaciones a 5.1).
- Falla si:
 - No existen clientes haciendo seguimiento de equipajes de ese vuelo
- Ejemplo de cliente de invocación: Desde la consola t3 finalizar el seguimiento de equipajes del vuelo de código AC1234 para dejar de recibir la información en la consola t1 (donde se invocó a 5.1 para ese vuelo) y en la consola t2 (donde también se invocó a 5.1 más tarde para ese mismo vuelo) mostrando el evento de finalización de seguimiento de equipajes en las tres consolas, y finalizando la ejecución de los tres clientes:

<pre>t1\$> sh track.sh -DserverAddress=10.6.0.1 :50051 -Daction=watchFlight -Dflight=AC1234 Started watching Flight AC1234 Assigned Pier #1 to Flight AC1234 Bag-Drop STARTED for Flight AC1234 with 3 passengers and up to 4 bags (100 kg) ... Stopped watching Flight AC1234 t1\$> _</pre>	<pre>t2\$> sh track.sh -DserverAddress=10.6.0.1 :50051 -Daction=watchFlight -Dflight=AC1234 Started watching Flight AC1234 Bag-Drop STARTED for Flight AC1234 with 3 passengers and up to 4 bags (100 kg) ... Stopped watching Flight AC1234 t2\$> _</pre>	<pre>t3\$> sh track.sh -DserverAddress=10.6.0.1 :50051 -Daction=unwatchFlight -Dflight=AC1234 Stopped watching Flight AC1234 t3\$> _</pre>
--	--	--

5.3 Equipajes cargados en aviones

- Funcionalidad: Consultar los equipajes que fueron trasladados a los aviones del aeropuerto (se invocó a 4.3). El orden de impresión debe coincidir con el orden en que los equipajes fueron trasladados. Opcionalmente se puede indicar el código del vuelo para filtrar el resultado. Para cada equipaje se debe indicar el *bag-tag* correspondiente (como se muestra en 2.3).
- No falla
- Ejemplo de cliente de invocación: Consultar los equipajes que fueron trasladados a los aviones del aeropuerto.

```
$> sh track.sh -DserverAddress=10.6.0.1:50051 -Daction=planeBags
###
Bag-Tag Details
-> Flight AC1234
-> Passenger DEF
-> Weight 21.0 kg
```

```
-> Bag-Tag #416
###
Bag-Tag Details
-> Flight AC1234
-> Passenger ABC
-> Weight 22.5 kg
-> Bag-Tag #412
###
Bag-Tag Details
-> Flight OS987
-> Passenger ZZZ
-> Weight 20.3 kg
-> Bag-Tag #413
###
```

Para filtrar el resultado y sólo listar los equipajes del vuelo de código AC1234:

```
$> sh track.sh -DserverAddress=10.6.0.1:50051 -Daction=planeBags
-Dflight=AC1234
###
Bag-Tag Details
-> Flight AC1234
-> Passenger DEF
-> Weight 21.0 kg
-> Bag-Tag #416
###
Bag-Tag Details
-> Flight AC1234
-> Passenger ABC
-> Weight 22.5 kg
-> Bag-Tag #412
###
```

5.4 Equipajes pendientes de carga en aviones (Sólo para grupos de 5 integrantes)

- Funcionalidad: Consultar los equipajes que todavía no fueron trasladados a los aviones del aeropuerto (no se invocó a 4.3 para esos equipajes). El orden de impresión debe coincidir con el orden en que se emitieron los *bag-tags* (esto es, el orden en que fueron aceptados en las estaciones de *bag-drop*). Opcionalmente se puede indicar el código del vuelo para filtrar el resultado. Para cada equipaje se debe indicar el *bag-tag* correspondiente y la ubicación del equipaje (como se muestra en 3.3)
- No falla
- Ejemplo de cliente de invocación: Consultar los equipajes que todavía no fueron trasladados a los aviones del aeropuerto.


```
$> sh track.sh -DserverAddress=10.6.0.1:50051 -Daction=pendingBags
###
Bag-Tag Details
-> Flight AC1234
-> Passenger ABC
-> Weight 22.5 kg
-> Bag-Tag #412
###
Bag-Tag Location
[ ] Baggage Room
[ ] Pier
[X] Cart #1
[ ] Plane
###
Bag-Tag Details
-> Flight OS987
-> Passenger ZZZ
-> Weight 20.3 kg
-> Bag-Tag #413
###
Bag-Tag Location
[X] Baggage Room
[ ] Pier
[ ] Cart
[ ] Plane
###
```

Para filtrar el resultado y sólo listar los equipajes del vuelo de código AC1234:

```
$> sh track.sh -DserverAddress=10.6.0.1:50051 -Daction=pendingBags
-Dflight=AC1234
###
Bag-Tag Details
-> Flight AC1234
-> Passenger ABC
-> Weight 22.5 kg
-> Bag-Tag #412
###
Bag-Tag Location
[ ] Baggage Room
[ ] Pier
[X] Cart #1
[ ] Plane
###
```

5.5 Dashboard de equipajes despachados de un vuelo (Sólo para grupos de 5 integrantes)

- Funcionalidad: Consultar el total de equipajes y kg correspondientes aceptados por las estaciones de bag-drop a partir del código del vuelo, indicando si están en el *baggage room*, el *pier*, en los *carts* o en el avión. También se debe indicar la franquicia total de todos los pasajeros del vuelo (información provista al invocar a 2.1)
- Falla si:
 - No existe un vuelo con ese código
- Ejemplo de cliente de invocación: Consultar el total de equipajes y kg correspondientes aceptados por las estaciones de *bag-drop* para el vuelo del código OS987.

```
$> sh track.sh -DserverAddress=10.6.0.1:50051 -Daction=dashboard
-Dflight=OS987
###
Manifest: 12 bags (276.0 kg)
###
Baggage Room: 0 bags (0.0 kg)
Piers: 5 bags (100.5 kg)
Carts: 4 bags (80.0 kg)
Plane: 1 bag (20.3 kg)
###
Total: 10 bags (200.8 kg)
###
```

5.6 Dashboard de *carts* (Sólo para grupos de 5 integrantes)

- Funcionalidad: Consultar el total de equipajes y kg correspondientes presentes en los *carts*. Opcionalmente se puede indicar el código del vuelo para filtrar el resultado. El orden de impresión es alfabético por código de vuelo y, para cada vuelo, ascendente por número de *cart*. No se deben incluir los *carts* vacíos.
- No falla
- Ejemplo de cliente de invocación: Consultar el total de equipajes y kg correspondientes presentes en los *carts*.

```
$> sh track.sh -DserverAddress=10.6.0.1:50051 -Daction=cartBags
###
Flight AC1234: 1 carts
###
Cart #1: 2 bags (43.5 kg)
###
Total: 2 bags (43.5 kg)
###
Flight OS987: 3 carts
###
Cart #2: 5 bags (100.5 kg)
Cart #5: 4 bags (80.0 kg)
```

72.42 Programación de Objetos Distribuidos - TPE1 2026 1C

```
Cart #6: 1 bag (20.3 kg)
###
Total: 10 bags (200.8 kg)
###
```

Para filtrar el resultado y sólo listar los equipajes del vuelo de código AC1234:

```
$> sh track.sh -DserverAddress=10.6.0.1:50051 -Daction=cartBags
-Dflight=AC1234
###
Flight AC1234: 1 carts
###
Cart #1: 2 bags (43.5 kg)
###
Total: 2 bags (43.5 kg)
###
```

Hechos y Consideraciones

Para simplificar el desarrollo se pueden tomar los siguientes considerandos como ciertos:

- No es necesario que tenga persistencia. Al reiniciar el sistema se comienza de cero la operación del mismo
- Los valores de los parámetros de los clientes no contienen espacios
- En todos los servicios, en caso de falla, se espera que el cliente capture el error en cuestión y muestre un mensaje apropiado en salida estándar.

Requisitos

Se requiere implementar:

- Todos los servicios utilizando **Spring gRPC 1.0.2** y **Java 25 LTS**, teniendo en cuenta que los servicios deben poder atender pedidos de clientes de manera concurrente y responder a lo indicado en este enunciado
- Los clientes indicados cada uno como una aplicación de consola diferente utilizando **scripts de bash** independientes

Muy Importante:

→ **Respetar EXACTAMENTE**

- ◆ **Los nombres de los *scripts* y sus parámetros** (Si se pide -Dcart no se aceptará -Dkart, -Dcarrito, etc.)
- ◆ **El orden y formato de salida enunciado**

→ **En TODOS los pom.xml que entreguen deberán definir**

- ◆ El name con la siguiente convención: "tpe1-gX-Z" donde X es el número de grupo y Z es parent, api, server o client. Por ejemplo:
<name>tpe1-g15-api</name>

Informe del Proyecto

Además de la entrega del código fuente, deberán incluir un informe breve estructurado en las siguientes tres secciones:

1. Decisiones de diseño e implementación de los servicios

Explica los patrones y estructuras utilizados en cada área del proyecto, fundamentando su utilidad y aplicación práctica.

Aclaración: No se busca un listado descriptivo de clases ni proyectos, sino una justificación clara de las decisiones técnicas adoptadas.

2. Criterios aplicados para el trabajo concurrente

Detallá las decisiones tomadas para garantizar que el sistema sea thread-safe:

- **Secciones protegidas:** ¿Qué partes del código se decidieron proteger y cómo se implementó dicha protección?
- **Estrategia y justificación:** ¿Por qué optaron por este enfoque en particular?
- **Alternativas:** Indicar si evaluaron otras opciones de concurrencia y por qué las descartaron.

3. Potenciales puntos de mejora y/o expansión

Posibles alternativas de implementación que no hayan llegado a explorar, o nuevas operaciones que aportarían valor al sistema. **No incluyan** sugerencias sobre persistencia en base de datos ni tests unitarios o de integración.

Uso de Git

Es obligatorio el uso de un repositorio Git para la resolución de este TPE. Deberán crear un repositorio donde todos los integrantes del grupo colaboren con la implementación. El repositorio debe crearse a partir del *template repository* indicando en Campus (usando la opción *Use this template -> Create a new repository*). No se aceptarán entregas que utilicen un repositorio git con un único *commit* que consista en la totalidad del código a entregar.

Los distintos *commits* deben permitir ver la evolución del trabajo, tanto grupal como individual.

Muy importante: **los repositorios creados deben ser privados, solo visibles para los integrantes del grupo y, solo en caso de que se lo solicite específicamente la cátedra.**

Material a entregar

Cada grupo deberá subir al **Campus** **ÚNICAMENTE UN ARCHIVO COMPACTADO** conteniendo:

- El **código fuente** de la aplicación:
 - Utilizando la misma estructura Maven vista en las clases 4, 5 y 6.
 - Con una correcta separación de las clases en los módulos *api*, *client* y *server*
 - **No se deben entregar los binarios.** Recordar de ejecutar el comando `./mvnw clean` antes de la entrega.
- Un **README.md** indicando cómo realizar el build, cómo correr el servidor y cómo correr cada uno de los cinco clientes (indicando todos los parámetros posibles con ejemplos de invocación)
- El directorio oculto `.git/` donde se encuentra la historia de commits y modificaciones. Recordar que la descarga desde github.com no incluye el directorio `.git/`.
- El **informe** ya mencionado en un archivo **Informe.pdf** ubicado en el directorio **doc/** **dentro de la raíz del repositorio**. Incluir una carátula indicando todos los integrantes del grupo.

Corrección

El trabajo no se considerará aprobado si:

- No se entregó el trabajo práctico en tiempo y forma
- No se entrega alguno de los materiales solicitados en la sección anterior
- El código no compila utilizando Maven en la consola (de acuerdo a lo especificado en el **README.md** escrito por el equipo)
- El servicio no inicia cuando se siguen los pasos del mismo **README.md**.
- Los clientes no corren al seguir los pasos del **README.md**.
- La entrega contiene un único *commit* que consista en la totalidad del código a entregar.

Si nada de esto se cumple, se procederá a la corrección donde se tomará en cuenta:

- Que los servicios y clientes funcionen correctamente según las especificaciones dadas
- El resultado de las pruebas y lo discutido en el coloquio
- La aplicación de los temas vistos en clase: Concurrencia y gRPC
- La modularización, diseño, testeo y reutilización de código
- El contenido y desarrollo del informe

Cronograma

- **Presentación del Enunciado:** miércoles 09/09
- **Entrega del trabajo:** Estará disponible hasta el jueves 24/09 a las 23:59 la actividad "TPE 1" localizada en la sección Contenido / Evaluaciones. En la misma deberán cargar el archivo compactado
- **No se aceptarán entregas pasado el día y horario establecido como límite**
- **El día del coloquio será el miércoles 30/09**
- **El día del recuperatorio será el miércoles 25/11**

Dudas sobre el TPE

Las mismas deben volcarse en los **Debates** del Campus ITBA.

- Crear un debate por pregunta.
- Revisar los debates previos para ver si la pregunta ya fue hecha.
- Revisar los debates para ver si alguna respuesta afecta a la implementación que están haciendo.

Recomendaciones

- Para el parseo de argumentos del cliente como `-Dcart=1` consultar [java.lang.System#getProperty\(java.lang.String\)](#)
- Para ocultar la salida por defecto de Spring Boot consultar las propiedades `spring.main.banner-mode` y `spring.main.log-startup-info`